NeoNEXUS: A Framework for Morphological Analysis of Density Fields

Ivan Spirov s4923405

January 8, 2026

Abstract

I present NeoNEXUS, a modular framework for multiscale morphological analysis of continuous scalar fields, designed as a flexible and extensible generalization of the MMF and NEXUS formalisms. NeoNEXUS preserves the core physical and mathematical principles underlying Hessian-based structure identification, specifically the use of eigenvalue signatures evaluated across a scale space, while introducing a software architecture that decouples feature definitions, filtering strategies, and thresholding schemes. Implemented in Julia, the framework leverages multiple dispatch and explicit caching of Hessian eigenstructures to enable efficient reuse across analyses to preserve computational resources. NeoNEXUS supports a range of smoothing filters, including Gaussian and logarithmic Gaussian variants, and is designed to accommodate both established and experimental threshold-selection techniques. Although motivated primarily by applications to cosmic web identification in cosmological density fields, the framework is intentionally domain-agnostic and suitable for extension to other areas of scientific and engineering data analysis. This report describes the conceptual design, mathematical formulation, and implementation details of NeoNEXUS, discusses validation strategies and known limitations, and outlines directions for future development toward a fully deployable analysis tool.

University of Groningen

HTSM Masterwork Project

Supervisor: *Rien van de Weygaert*

Contents

1 Introduction **3**
- 1.1 Context and Motivation . . . 3
- 1.2 NeoNEXUS Overview . . . 3
- 1.3 Structure of This Report . . . 4

2 Project Objectives **4**
- 2.1 Primary Objective . . . 4
- 2.2 Secondary Objectives . . . 4

3 NeoNEXUS Formalism **5**
- 3.1 Mathematical Foundations . . . 5
- 3.2 Conceptual Pipeline . . . 7
- 3.3 Assumptions and Constraints . . . 9

4 Historical Context **10**
- 4.1 Previous Implementations . . . 10
- 4.2 Limitations of Legacy Methods . . . 11
- 4.3 Evolution Toward NeoNEXUS . . . 12

5 Methodology and Implementation **13**
- 5.1 Software Design . . . 13
- 5.2 Implementation Details . . . 14
  - 5.2.1 Hessian . . . 14
  - 5.2.2 Features . . . 15
  - 5.2.3 Filters . . . 15
  - 5.2.4 Thresholds . . . 16
- 5.3 Validation Strategy . . . 16

6 Discussion **16**
- 6.1 State of NeoNEXUS . . . 16
- 6.2 Convergence to NEXUS . . . 17
- 6.3 Comparison with Sister Project . . . 17
- 6.4 Further Planned Improvements . . . 18

7 Potential Use Cases **18**
- 7.1 Astronomical Applications . . . 18
- 7.2 Engineering Applications . . . 20
- 7.3 Medical and Biomedical Applications . . . 20
- 7.4 Future Extensions . . . 21

8 Conclusion and Recommendations **21**
- 8.1 Summary of Contributions . . . 21
- 8.2 Recommendations for Future Work . . . 21

1 Introduction

This report outlines my work on the NeoNEXUS project, its relevance in astronomy and beyond as well as all relevant components to it, including past implementations. Due to the program itself not being published in an official capacity yet, the full code is not made available, although results, plots and high level implementation will be outlined.

1.1 Context and Motivation

In the current scientific era a new, almost paradoxical issue arises - too much data to process. This is commonly referred to a Big Data. While in past epochs scientists had to make do with limited and statistically weak sets of data, contemporary scientists are practically overwhelmed with datasets spanning terabytes computationally and several orders of magnitude physically.

An example for that comes from my home field - astrophysics: with new survey results from state of the art telescopes arriving, we are faced with hundreds of millions of data points from each. For example, the Gaia mission's total catalogue boasts a number of stellar sources on the order of $\sim 1.7 \times 10^9$[5]. Similarly, the DESI mission promises to release a deep cosmological catalogue of tens of millions of galaxies with precise redshifts across a large fraction of the sky by the end of its nominal five-year survey and is expected to have measured redshifts for over 3.5×10^7 galaxies [4]. This is not to even mention the incredibly deep and precise world of astrophysical simulations, which form the backbone to a lot of modern research.

It is because of this state of overwhelming data that the need for specialized tools and frameworks arises, cutting down "manual" work by researchers. Recently, pushes in AI have been grabbing headlines because of the promise neural networks show in dealing with Big Data, but this project tackles something more simple and elegant - a formalism developed from an analytical basis, which requires no training step and saves computation not only during deployment, but also during preparation[1]. It is no wonder then that the idea for this program originates from medicine - a field where manpower is a scarce resource and big data optimizations can save lives. This will be elaborated on in Section 4.

1.2 NeoNEXUS Overview

NeoNEXUS is the third generation of the MMF/NEXUS formalism - morphological density analysis tools. At the fundamental technique behind the algorithms is hidden in the acronym - Multiscale Morphology Filter. Given a density field, NeoNEXUS can detect predefined morphological features on multiple scales. This is invaluable in cosmic web analysis, but even in its conception is a multi-disciplinary idea, having originated in medical data analysis. The NeoNEXUS framework stands ahead of its predecessors in two major features embedded in its design philosophy - modularity and applicability. NeoNEXUS is usable within any context - from cosmology to engineering, provided an expert from the field extends the core functionality. Even within a single field of use, NeoNEXUS can be used for a wider range of purposes, due to the user's simple ability to switch features. This is done via abstraction - morphological features,

[1] That is to say there is no precomputation once the framework is prepared.

filters and thresholds are defined in an easy to extend and change way, allowing a truly immense range of use cases.

1.3 Structure of This Report

This report is structured as following: In Section 2, I will expand on the objectives of NeoNEXUS, the core vital features as well as further implementation goals. In Section 3, I will outline the NeoNEXUS formalism at its core, explaining the mathematical theory behind it. In Section 4, I will discuss the previous implementations of MMF/NEXUS, noting how they evolved into this project and the shortcomings that have been fixed. In Section 5, I will show the specific implementation, demonstrating how the unique architecture facilitates our goals. In Section 6, I will discuss the state of the NeoNEXUS project and note comparisons with a sister project I am collaborating on. In Section 7, I will delve into potential further use cases, both in science and beyond. Finally, in Section 8, I will give my conclusion, summary and outlook for further work.

2 Project Objectives

The objectives of this project break into roughly three categories: first, the core functionality, which must be reached for this project to even compare to its predecessors; second, the software improvements, which set this project apart; and, finally, numerical improvements such as performance, both in computational speed and memory requirements. The former is outlined in the first subsection, while the latter two are bundled into the second one, since their implementation is closely linked.

2.1 Primary Objective

At its core, NeoNEXUS must be able to perform the same functions as MMF and NEXUS, while also expanding on them. That means, when applied to cosmological data[2], NeoNEXUS must be able to correctly identify point-, line-, and sheet-like features on multiple scales (MMF) and do so robustly with specialized scale binning (NEXUS). In this context, point-like objects are very dense galaxy forming regions[3], line-like features are cosmic filaments - enormous structures, containing a majority share of the universe's mass responsible for immense tidal forces[3], and sheet-like features are cosmic walls - large objects vital to the evolution of large-scale structure.

It is without a doubt that the correct identification of these features is vital to the study of the cosmic web, not the least because a large share of the work done on cosmic evolution was done with the MMF/NEXUS algorithms. As a primary objective, therefore, we must ensure that NeoNEXUS is up to standard with these well-studied iterations.

2.2 Secondary Objectives

While the importance of NeoNEXUS' cosmological performance can not be ignored, it would be remiss to focus only on that. The core MMF formalism can easily be applied to several

[2]Note that this is often done to idealized simulated data. One of the motivations behind the expansive nature of NeoNEXUS is application to novel data such as gas (rather than dark matter) and observational data.

fields external to astronomy. This is most clear by its original entry into astronomy via Miguel Aragón-Calvo's 2007 paper[1], which adopted the scale space approach into astronomy from medicine. However, the potential applications in other fields don't end with medicine. This will be discussed further in Section 7.

All this is to say, the NeoNEXUS framework must, in addition to performing cosmological tasks, also have the groundwork laid for users from other fields to easily extend and apply the already existing framework. To this end two sets of changes can be made: interface abstraction and programming abstraction.

Interface abstraction means that the user end of the program (either in terminal, standard library or even GUI as developed in sister project) is agnostic about the data it is given. For example, our code should not care for astronomical units - it should find densities on scales of voxels and pixels, not parsecs or meters. That way a user can appropriate it for their own specialized use case. Similarly, we want to avoid using any proprietary data storage files or systems that aren't available to the average user. In the long run, fully portable versions for deployment can be given (again, as is done in a sister project).

Programming abstraction means that reoccurring and non-fundamental concepts and structures can be bundled into a larger abstraction. For example, instead of defining functions for nodes, filaments and walls and then defining a new function for a new cosmological object, we can simply define a MorphologicalFeature abstraction, which carries all common behaviors and fields and is then inherited by individual morphological features. This way the framework can very easily be extended, debugged and worked with.

Fulfilling these secondary objectives is really the main distinction from the previous formalisms, other than writing NeoNEXUS in a more modern programming language. Additionally, it is also what truly makes this project an appropriate HTSM Masterwork project. As such, "secondary" here doesn't mean optional - it simply means a broader criterion. It is also harder to say the secondary objective is fully accomplished as there will always be further expansion of the project, while fulfilling the primary task is really manifestly required.

3 NeoNEXUS Formalism

The NeoNEXUS formalism is a direct descendant of the MMF and NEXUS formalisms as laid out by Miguel Aragón-Calvo[1] and Marius Cautun[2], respectively. While NeoNEXUS aims to extend these frameworks, the following section will explain the fundamentals of the MMF and NEXUS operations. Understanding these foundations is essential, as NeoNEXUS preserves their core philosophy while abstracting and generalizing several of their assumptions.

3.1 Mathematical Foundations

At the heart of both the MMF and NEXUS approaches lies the representation of a physical system as a continuous density field. In an astrophysical context, this density field is typically denoted as $\rho(\mathbf{x})$, or equivalently as the dimensionless overdensity field

$$\delta(\mathbf{x}) = \frac{\rho(\mathbf{x}) - \bar{\rho}}{\bar{\rho}}, \tag{1}$$

where $\bar{\rho}$ denotes the mean density. Such a field may be constructed from discrete tracers, such as galaxies or simulation particles, using interpolation or smoothing techniques. One such formalism is the Delaunty Tesselation Field Estimator (DTFE), which works particularly well for MMF/NEXUS due to its conservation of small-scale details. A modernized version of DTFE is also in the works as a supplementary project to this one.

Morphological information is extracted by analyzing the local curvature of the density field through its Hessian matrix,

$$H_{ij}(\mathbf{x}) = \frac{\partial^2 \rho(\mathbf{x})}{\partial x_i \partial x_j}, \tag{2}$$

or equivalently through the Hessian of the smoothed overdensity field. The eigenvalues of the Hessian encode the local second-order structure of the field and provide a natural means of classifying regions according to their morphology. This approach is closely connected to the Zel'dovich approximation[8], in which the eigenvalues of the deformation tensor govern the anisotropic collapse of matter along principal axes, leading to the formation of walls, filaments and nodes in that order.

This is directly related to how MMF/NEXUS classifies regions via Hessian eigenvalues. Regions where none of the eigenvalues suggest compression are assumed to be voids. Regions with one minimized eigenvalue are associated with collapse along one axis, in other words a wall. Similarly, two small and one dominant eigenvalues are associated with filaments and finally collapse along all three dimensions is associated with clusters. This eigenvalue-based classification provides a physically motivated elegant connection from the density field to the emergent large-scale structure.

The major innovation introduced originally by MMF is the incorporation of a multiscale formalism. Rather than analyzing the density field at a single smoothing scale, the field is convolved with a series of filters characterized by a scale parameter R,

$$\rho_R(\mathbf{x}) = \int \rho(\mathbf{x}') W_R(\mathbf{x} - \mathbf{x}')\, \mathrm{d}^3\mathbf{x}', \tag{3}$$

where W_R denotes a smoothing kernel, commonly Gaussian. The Hessian and its eigenvalues are then evaluated at each scale, allowing structures of various sizes to be detected without larger structures dominating the smaller ones.

The NEXUS formalism introduced several refinements beyond the original MMF framework. This includes log-Gaussian filtering (NEXUS+), which improves sensitivity to structures spanning a wide dynamic range in density, as well as additional algorithms designed to enhance robustness against noise and sampling effects. Furthermore, NEXUS emphasizes a more unified treatment of different morphological components, enabling consistent classification across nodes, filaments, walls, and voids within a single framework. Along with the standard NEXUS and NEXUS+ formalisms, a family of algorithms was also introduced such as Tidal NEXUS and NEXUS Sheer, which make use of the velocity field[3].

NeoNEXUS further builds on top of MMF and NEXUS by constructing a generalized framework, which can not only converge to any of the previous algorithms, but also be used in con-

[3]For the purposes of brevity, these will not be outlined in this report, but note that due to NeoNEXUS' modular nature, they are easily introduced, in addition to any other features contingent on an auxiliary field.

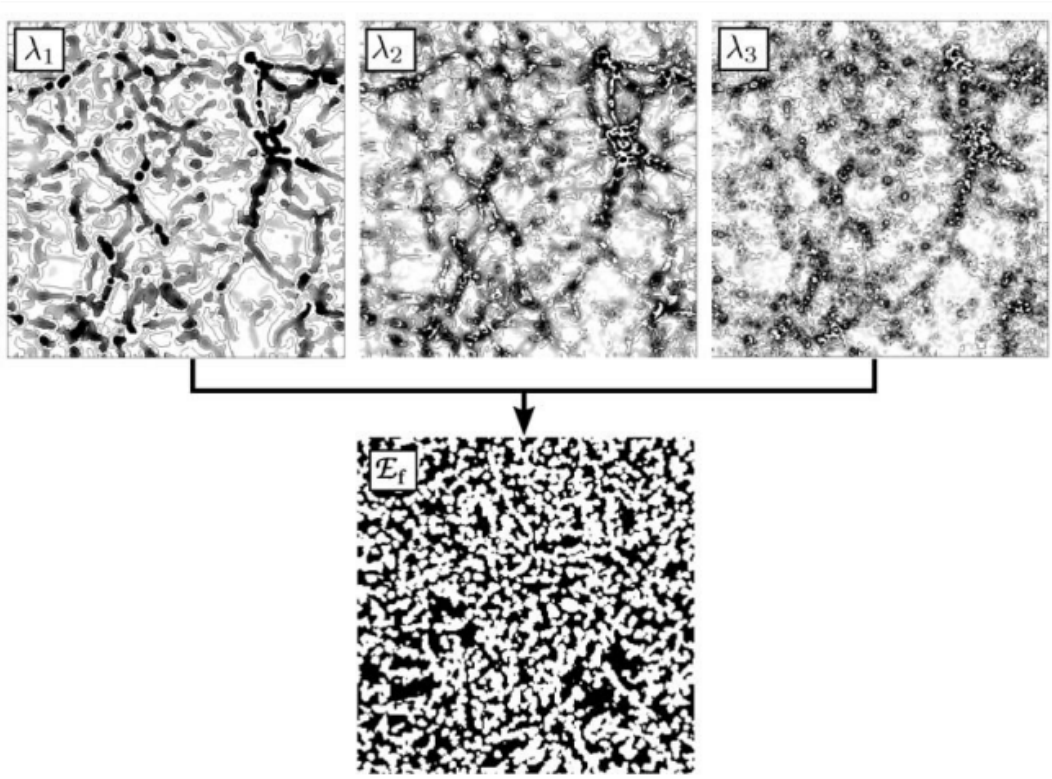

Figure 1: Eigenvalue maps taken from Figure 4 in Aragón-Calvo et al 2007 [1]. Demonstrates how eigenvalue fields come together in creation of morphology mask - original part of signature identification.

structing new and improved versions in a very approachable, testable and modular method.

3.2 Conceptual Pipeline

Within the inner workings of MMF/NEXUS, there are four fundamental types of fields to be understood, all deeply related. The first is the input density field. This a three dimensional field where some multiscale structure can be found. Second is the Hessian, more notably the three eigenvalues, which form a family of three fields, vital to our analysis as outlined above. An example for the Hessian eigenvalue fields can be seen in Figure 1.

Third is the signature field, which is the core of MMF/NEXUS and the best analytically defined part. The signature field is essentially an answer to the question "How likely is the presence of a certain feature?". For any given feature MMF/NEXUS uses, a signature field is generated using the aforementioned Hessian eigenvalue test at each scale. After this is done on all scales, the scaled signature fields are combined to give one final signature field. An example for a signature field can be seen on the second row of Figure 2. Notice the continuum of values - each voxel in the field has a given probability to be the desired feature. It can crudely be thought of as a non-normalized statistical probability.

The final field we look at is the threshold map. Since MMF/NEXUS seeks to give an answer to which regions are dominated by a filament, node, wall or void, we must give a concrete answer instead of a probability. That is why the final step for each feature is thresholding. Given a certain signature map, a final step is taken to find the threshold value - the signature

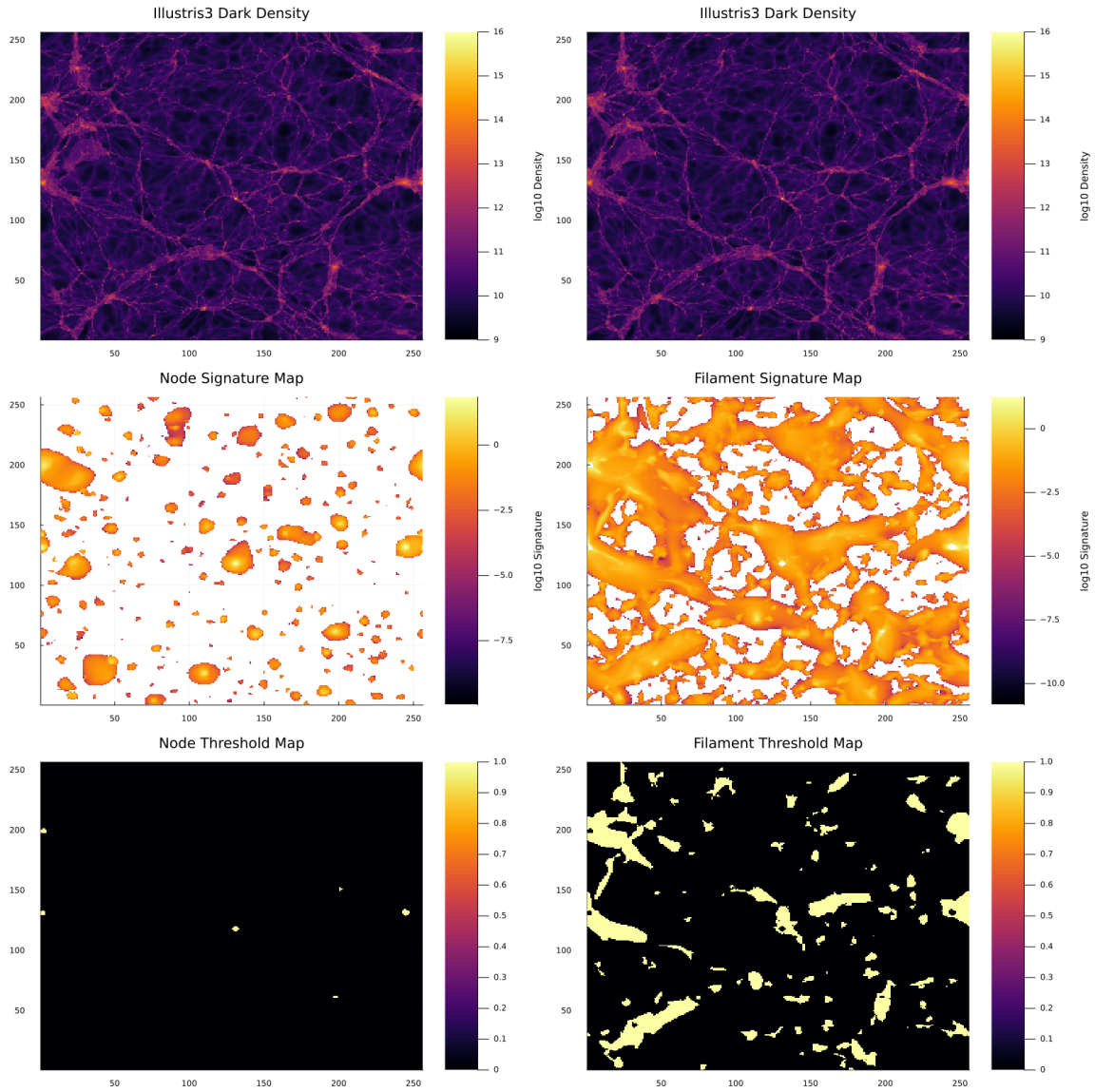

Figure 2: 2D slice of example maps for MMF/NEXUS on Illustris3 Dark. Left side shows maps for nodes and right for filaments. First row shows underlying density, second row shows signature maps and the third row shows threshold maps after applying a cutoff. Notice filament maps have nodes removed due to ordering of identification.

"probability" above which we accept the feature and below which we reject it. This is done to both remove noise (since random collapse-like structure can happen based on probability, rather than physics) and remove insignificant structures that haven't collapsed yet. The precise best way to define the thresholding point is the subject of ongoing research and we hope NeoNEXUS will greatly aid in that. Only for nodes is there an analytical motivation - using the spherical collapse model, we consider that clusters should have an average density 370 times greater than that of the average universe. The current way it is defined procedurally is via the NEXUS implementation[2] by finding the signature point where half of the potential nodes are allowed[4]. Filament and wall identification follows a similar technique, but since there is no spherical collapse model equivalent, the threshold value is varied until a maximum point for the mass derivative as a function of threshold value can be found. Both of these procedures can be seen in Figure 3.

The physical density field, signature map and threshold map can be clearly seen on Figure 2. Notice that the signatures (especially in log space) can be much larger and more extended than how they appear visually in the cosmic web, which is why the thresholding step is crucial. It should be noted that the order of feature detections matters - in the MMF/NEXUS case, it is imperative that nodes are found first, then filaments, then walls and finally the leftovers are considered voids. This is because we use collapse criterion along dimensions. If a node has covers the criterion for collapse over three dimensions, then naturally it will also cover the criterion for two and one. Therefore, we first detect nodes, then tag those voxels as zero signature for filaments and walls, after which we do the same for filaments and finally walls are thresholded based on the remains of the other two. This can be seen in the lower right panel of Figure 2, where holes can be seen in the filaments, since there we have clusters.

This process outlines the original MMF/NEXUS routines. While some points are essentially analytical and can be kept as is, there is clear room for improvement in some of the steps. However, instead of delving into a more precise method for finding a threshold point, this work aims to develop the framework to easily and demonstrably find better threshold techniques and approaches.

3.3 Assumptions and Constraints

The MMF/NEXUS formalism is not without issues. I will outline two classes of issues - fundamental to the formalism and present in the formulation. The former will inevitably be inherited by NeoNEXUS and can only be mitigated with secondary routines, while the latter will ideally be removed altogether. The largest issue in MMF/NEXUS is one present in most cosmological computation - Fourier space. Since a lot of the procedures and routines required for our calculations are done in Fourier space for computational ease[5] they are also plagued by the issues Fourier space brings. Since Fourier representations of functions expect periodic behavior, often fields transformed into Fourier and back are also expected to have so-called periodic boundary conditions. If the periodic boundaries are not covered, noise and unwanted patterns are introduced. In principle NeoNEXUS allows for the replacing of all Fourier-dependent routines with

[4]While not fully phisically motivated, this is based on the maximum point of the mass gradient as a function of signature.

[5]One example is convolution (or smoothing), which in Fourier space is simple multiplication.

alternatives, but at least on this stage, it appears computationally infeasible[6].

Thankfully, the most glaring issues and biases in the original MMF/NEXUS implementations can be removed via NeoNEXUS. Note that NeoNEXUS itself does not remove these issues outright - it introduces the framework and tools to easily find alternatives or to look into whether or not the already existing routines are truly optimal. This includes, as mentioned previously, the ability to define any number of morphological features as well as ones dependent on auxiliary fields. NeoNEXUS also abstracts the thresholding routines used, meaning one could use the already existing node, filament and wall detections, but they could also chose to define their own and test them against each other. MMF/NEXUS is already very robust as a formalism, which means that now, when implemented in a more modular manner, it can also smooth out all former issues.

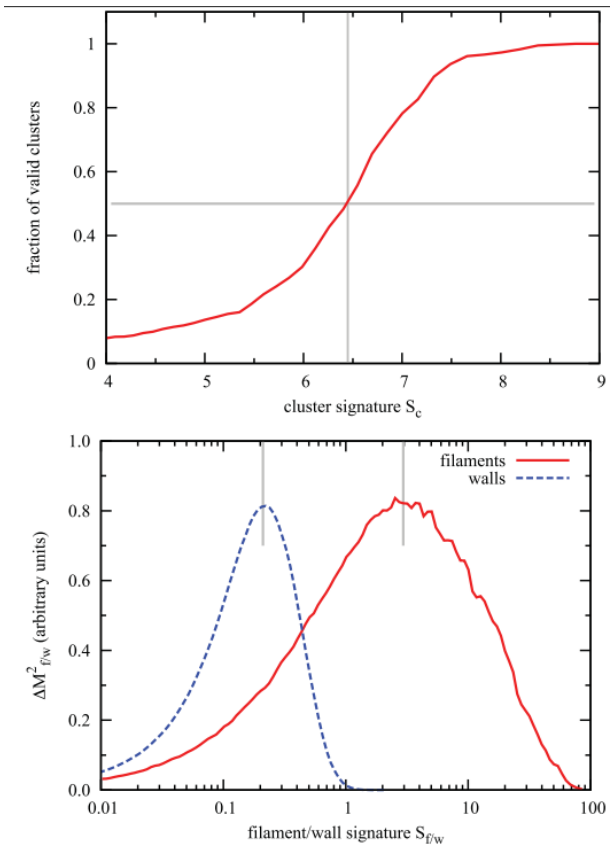

Figure 3: Example NEXUS thresholding as shown in Figure 3 of Cautun et al 2013 [2]. While point of maximum derivative is somewhat mansifest by imposing a half mass requirement for clusters, in filament and wall thresholds it needs to be numerically determined.

4 Historical Context

As outlined in previous sections, NeoNEXUS is the third generation of multiscale morphological feature extraction software. In this section I will briefly outline the previous iterations of this software and explain the improvements NeoNEXUS brings.

4.1 Previous Implementations

While the uses of scale space and Hessian shape finding can be traced back to the 1990s with works like Sato et al 1998[7], the start of the MMF/NEXUS family of programs was set in 2007 with the introduction of MMF into cosmology by Miguel Aragón-Calvo et al.[1]. The original MMF formalism focuses greatly on the scale space approach and the idea that to truly properly analyze the cosmic web, one must first break it into scale bins (in term introducing a fourth, scale dimension) and analyze those smoothed densities. This original implementation uses 9 individual fields as a part of the classification of the field. In fact, the routine as described in the paper doesn't aim at mapping a density field as much as it aims to classify individual particles. In other words, the input is really a scattered set of data that is transformed into a density field using DTFE, the resultant field is taken into MMF and finally the original objects are classified as belonging in nodes, filaments, walls or voids. This approach is inherently more observation-

[6]It is still possible however and a future work using NeoNEXUS could explore exactly that.

friendly. In a real world application of this software, one does not begin with a well-defined density field of the universe.

The second, more modernized iteration of MMF came in 2012 with the introduction of NEXUS by Marius Cautun et al[2]. NEXUS was the better formalized, more explored version of MMF, converging to MMF when needed, but also introducing more robustness and new implementations. The first major change that came with NEXUS was NEXUS+, which made one simple, elegant change - analyzing the logarithm of the smoothed density fields, rather than the original field. This change, while introducing computational complications, made the detection of filaments and walls much better. Node finding is still done better using the classical method, but since the introduction of NEXUS+, it is the primary method used for filaments and walls, resulting in sharper features and better defined cosmic web. The other big addition that came with NEXUS was the velocity family of NEXUS algorithms - NEXUS_veldiv, NEXUS_tidal and NEXUS_velshear. These iterations of NEXUS ran with velocity data for the cosmic web, allowing for the detection of specific patterns and features, related to the motions within the cosmic web. The NEXUS implementation really resembles a more mature, well defined formalism becoming ready to use as a wider research tool.

4.2 Limitations of Legacy Methods

It is manifest in the existence of this report that there are limitations in MMF and NEXUS, since, if they were already perfect, this project would be moot. The flaws in the past two iterations break into roughly three categories, although they are deeply entwined: ease of use, flawed implementation and non-generalized use. I will explain the first from an astronomical use case, the second from a programming perspective and finally, the last one as a general user from another field.

The ease of use issue is problematic for astronomers, hoping to run NEXUS for cosmic web analysis. The current code for NEXUS is not widely available, but also isn't very intuitive to work. It was originally developed in C++ for the purposes of high efficiency computing, but as a result it can be intimidating and difficult to use for astronomers who might only be familiar with Python. Additionally, there is no simple distributable compilation for a user to open - the full script must be installed and the user needs to ensure all required libraries are available and properly linked in the makefile before installing. This can already be a major filter for potential users, but even if they go through the process of installing everything, NEXUS can be very difficult to run. The best way to run the program is through the command line, giving parameters as additional instructions in one long command, including vital settings for feature finding. For better ease of use a Python script can be written to run that program, but that interface still isn't very user friendly. The only such available script has all the settings hard coded in. An ideal NEXUS implementation, at least for astronomers, would have a Graphical User Interface (GUI) and a single deployable file[7] that can be taken and used out of the box for any astronomer. The issue is so pressing, in fact, that as a sister project to this one, a more direct NEXUS implementation[8], is being developed with a GUI and deployable in mind.

[7]Three actually, since there must be separate ones for Windows, Linux and Mac.

[8]By direct, I mean this implementation does exactly what original NEXUS does with no functional changes.

Issues related to flawed implementation can be varied and, especially in the realm of software engineering, can be a matter of style or preference. However, if we are to treat this product not only as a black box app that simply does what is needed, but also as a product we can extend to match our uses, then we must ensure the codebase is clean and well documented. As it stands now, NEXUS has no software support and has received no updates or changes for years. An ideal NEXUS implementation would be made in such a way that not only the user can run the code, but a developer can read through it, modify and extend it. This is one of the major issues that will be fixed in NeoNEXUS.

Finally, the issue of non-generalized use comes up. As has been outlined in the motivation for this project and as it will be outlined further in Section 7, NEXUS has the potential to be applied to many other fields. The formalism itself isn't inherently cosmological and can serve just as well in many other contexts. However, with the current way it is developed that is close to impossible. For one, the data files that are accepted are proprietary astronomical ones, meaning a user from another field will struggle greatly to use it. Second, related to this, NEXUS works in parsecs as a baked in unit - despite the data existing in voxels, metadata in the files converts each voxel length to physical length and NEXUS operates on that, meaning that a very motivated external professional would have to scale their data to cosmological sizes and convert back at the end. These generalization limitations are one of the main motivators behind NeoNEXUS.

4.3 Evolution Toward NeoNEXUS

NeoNEXUS is named this way for a simple reason - it is not a fundamentally different formalism. All of the logic within MMF/NEXUS is sound and works incredibly well when applied correctly to cosmological data. NeoNEXUS is simply the new generational implementation of this. The chosen programming language is Julia - a modern, high efficiency language that is easy to read, maintain and use, even by programmers less versed in computer science. The core motivation in NeoNEXUS is abstraction - breaking all of the functions and concepts into more general objects that can easily be extended. This has two merits - a strictly scientific astronomical one and a general one.

First, as astronomers we find flaws in the existing formalism and leaps in motivation behind certain decisions. However, implementing a second NEXUS that has one or two changes and testing both to see which one works better is inefficient and tedious. Instead, the goal is to create one general framework where single features can be swapped out and tested in order to find the best ones for a given use case. This means that apart from being the state of the art MMF/NEXUS implementation for tracing the cosmic web, NeoNEXUS will also serve as a computational research tool to find exactly how the inner workings of MMF/NEXUS slot into each other and what improvements give better answers. In short, apart from being more pleasant to use, NeoNEXUS opens the door to many further research projects.

The second merit to generalization is also what justifies NeoNEXUS being a proper HTSM Masterwork project. As stated numerous times so far, the formalism behind MMF/NEXUS can logically be adapted into many other fields. However, if at present a researcher from another field, medical analyst or engineer wants to use NEXUS, they would have to either do a convoluted series of steps to force astronomical software to work for their purposes or write their own. In both

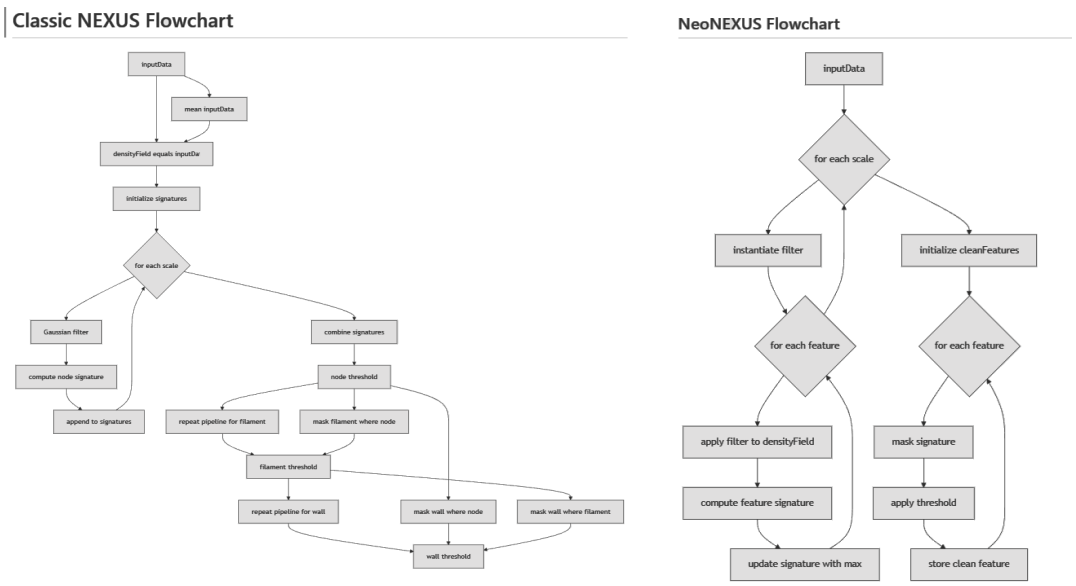

Figure 4: Comparison Flowcharts of Classic NEXUS and NeoNEXUS. Despite having more points, classical NEXUS supports fewer individual morphological features. Notice loops over features in NeoNEXUS - because of abstraction, the same repeated steps are not done manually.

cases they have to do undue work. Instead, when equipped with NeoNEXUS, this hypothetical user can quickly and efficiently extend the framework for their own uses. Ideally, this transforms NeoNEXUS into an ace of all trades.

5 Methodology and Implementation

This section will provide an explanation for the full NeoNEXUS framework and its implementation in Julia. Because the codebase is not published yet, the exact code will not be given, but a rundown of the design, structure, algorithms and data structures as well as tests and benchmarks will be given here.

5.1 Software Design

The fundamental steps within MMF/NEXUS are unchanged in the NeoNEXUS implementation. The major difference is the ordering and exact implementation. A pair of flowcharts in Figure 4 shows the compared workflow of both implementations. Note that for clarity in the classical NEXUS flowchart, filament and wall loops are not drawn[9]. The NeoNEXUS flow appears simpler because when features are abstracted, an arbitrary number can be added without an increase in complexity for the overall program. Note that initialization steps for NeoNEXUS, such as definition of features and thresholds is not included since they can be user defined.

The workflow of NeoNEXUS is separated into two major loops, as can be seen on the right side of Figure 4. The first loop creates the signatures for each feature and the second the clean threshold maps from them.

In the first loop, at every scale a filter is instantiated. Note that filters act differently on

[9]This means that if a full version of the algorithm was drawn, the flowchart would be even more convoluted.

different features (for example log filter for NEXUS+ or linear filter for nodes). The filter is then applied to the density field for each feature, noting that it can act differently as a function of feature. This is essentially the scale space step. A signature is then computed as a function of the given feature and that signature is compared with the feature's global signature map. This is repeated for each feature with the ability to preserve state between loop iterations via a cache, explained further below. Once the first loop is complete, a full signature map is created for each feature.

The second loop once again loops over each feature, but this is after we have collapsed the scale space, so it is simpler. The ordering of features is important here as it was in NEXUS originally, but this time it isn't hard coded. Each feature first masks its own signature via the previous features' threshold maps - for examples filaments cut out nodes as shown in Figure 2. Only then is the threshold function for the given feature called, giving the clean feature. The loop is then repeated for the next feature, which will mask by our new map as well as all the previous ones. This can happen over an arbitrary number of features. At least in the current implementation, non-mutually exclusive features are not calculated in the same run of the program. If there are such features, they can be run in separate batches. At the end of this loop, the program is finished and outputs the threshold maps.

5.2 Implementation Details

In this subsection I will note all of the specific custom data structures and behaviors made for the NeoNEXUS implementation. Ideally, this section should serve as a replacement to the programming code. Note that some of the structures and patterns are due to the strengths or limitations in Julia as a non Object Oriented Programming Language (OOP). The below subsections are dedicated to each vital part of the code. Within the code structure they correspond to .jl files used in the NeoNEXUS module.

5.2.1 Hessian

The first part of the NeoNEXUS framework is the Hessian computation core. The major structure do explain here is the HessianEigenCache - a helper structure used throughout the code. Since for a given density field, scale and filter the Hessian eigenvalue fields don't change, this structure acts as a helper to store these fields between features. In the NeoNEXUS implementation, we apply a singleton pattern and keep the cache as a single object instead of generating new ones. This is done to conserve memory, but should the need arise for two parallel caches to be maintained, this is possible[10]. The addition of the cache limits the number of expensive computations.

The rest of the Hessian framework is rather simple - functions that compute the Hessian eigenvalues and store them in a cache. The functions have mutating and nonmutating variants in case there is or isn't a cache present. The standard function computeHessianEigenvalues! mutates (as the Julia notation exclamation point dictates) the cache that is passed to it, while the computeHessianEigenvalues version does not expect a cache to be passed and instead returns a newly generated one.

[10]This would be a very unusual setup, but is possible if there are four or more alternating features that need to preserve memory from the first to the third and second to fourth.

5.2.2 Features

The word feature can be rather abstract and that is the general point. The most abstract structure is the AbstractFeature, which currently is inherited by only one child structure - AbstractMorphologicalFeature. In the future non-morphological features with different behaviors might be added, but currently only morphological ones exist. This abstract feature is in turn inherited by three child features - SheetFeature, LineFeature and NodeFeature. As a Julia structure, each morphological feature has a significanceMap, responseMap and kx,ky,kz parameter for Fourier operations. Since each feature is a singleton – only one NodeFeature is instantiated for a given input dataset – we use them to store the significance and response maps. Each of them also has a constructor function handled through overload that instantiates the two maps and fills the ks.

The computation of significance maps is done via the functor pattern in Julia[11]. When a user creates a `node = NodeFeature(...)` and then calls it via `nodeSignificance = node(density)` they receive a significance map by "calling" the object itself. Under the hood the functor accepts a field as well as a preexisting HessianCache, in case a previous feature had the same scale and filter, as well as an enum stating how the cache should be treated. If no cache is given or the enumerator is set to None, the function computes the eigenvalues as normal. If a cache is given and the mode is set to Read, the value is directly loaded, while if it is on Write, the eigenvalues are computed and written into the cache via the aforementioned mutation.

Once the functor has either loaded or computed the eigenvalues, it calls the computeSignature function. It should be noted that all AbstractMorphologicalFeature instances share the same functor, since the steps listed above are the same. Meanwhile computeSignature is different for each feature and this is handled through multiple dispatch - depending on what feature is given in the computeSignature function call, a different function is used. This links the signature to the feature, just as it naturally flows from the definition of a feature.

5.2.3 Filters

Similar to features, filters are handled via structures with functor calls. Currently the only filter is the Gaussian filter, so this section will outline the specific example for it. The structure GaussianFourierFilter inherits the general AbstractScalarFilter. The structure itself contains supplementary information for the Fourier calculations such as k vectors. The filtering process itself is handled with the functor pattern - once a filter is constructed, it can be called with a density field, scale and optionally a feature - `gauss(dens,R,feature)`. As mentioned previously, the same filter may act differently for different features, so the functor calls are build with multiple dispatch - calling with only density and scale gives the default option, calling with the same and a node struct also gives the default option, but if a wall or filament is given, a different function call for NEXUS+ is used. Finally, regardless of specific function call, a smoothed out density field is returned.

It should be noted that in the case of NEXUS+ there is an introduction of anti-optimization as far as computational speed is concerned. The steps of smoothing and Hessian computation can normally be combined in one swift, robust computation in the case of a standard Gaussian

[11] In OOP terms it can be thought of as a caller method.

filter via a predefined Hessian kernel function. This is not possible for NEXUS+ and so the steps as outlined above must be followed.

5.2.4 Thresholds

The final step is finding an optimal threshold for a given signature and feature. Since the precise way to determine an optimal threshold is the field of research, this has not been a neatly abstracted. Instead, several threshold functions have been designed and handled through multiple dispatch as before. The threshold function accepts a signature map and a morphological feature and depending on the feature given, a different routine is used. As explained in the original NEXUS paper[2], for nodes, we simply calculate the point at which half of our objects have a density larger than the analytically determined 370 times ambient. For walls and filaments, we instead have to manually determine the maximum of the derivative of the cumulative mass with respect to the signature. As further work is done using NeoNEXUS, better threshold functions will be found as is discussed briefly in Section 8.2.

5.3 Validation Strategy

Because of the scale of the NeoNEXUS framework, it is easy to get lost in complexity or permit compounding errors in the system. This is why in order to ensure the best quality possible, rigorous unit testing was implemented for each of the aforementioned elements of NeoNEXUS. Only after ensuring that each of them performs as expected compared to analytical examples were they combined and tested together. Further testing is ongoing, especially in specific examples such as the Illustris simulations. The results for NeoNEXUS are constantly compared to the results of MMF and NEXUS from literature as well as sister projects to ensure proper cosmological insight. Finally, speed tests and optimizations are done via Julia's built in benchmarking tools. With appropriate accuracy and precision, the final goal is improved computational efficiency.

6 Discussion

In this section I will discuss the present state of NeoNEXUS as a wholistic framework, its convergence to the original MMF/NEXUS software as well as its relevance compared to the current in-the-works sister project. Finally, considering the room for further work on the framework itself, I will outline further plans on the NeoNEXUS project beyond the HTSM Masterwork timeframe.

6.1 State of NeoNEXUS

NeoNEXUS, seen as a complete and deployable application, is still unfinished. While all the pieces for the final product are in place and the design is made, as explained in Section 5, a complete "deliverable" product is not avaiable. This is because the complete NeoNEXUS framework simply requires much more time than the HTSM Masterwork project can contain. Officially, the Masterwork is accredited as 5 ECTS, meaning roughly 140 hours spent on the project. However, even at this incomplete stage, NeoNEXUS has gone over that "budget". That

is to say, the framework established thus far as well as the overall design should stand as a Masterwork project, despite the final product still being in the works. At the end of this section, I will discuss further plans on the roadmap to completion of NeoNEXUS, which we hope will be finished by late March in time for the framework to be available to Bachelor students.

6.2 Convergence to NEXUS

As stated in Section 2, the primary objective of NeoNEXUS that needs to be covered to even consider this an MMF/NEXUS project is convergence to the original code. This was one major motivating factor behind the rigorous unit testing included in NeoNEXUS. The present code, in its implementation and significance mapping, follows the steps of MMF/NEXUS closely. The step of thresholding is still in open discussion, particularly on how zealous we want to be with the cleanup of insignificant structures. By the text of the papers introducing MMF and NEXUS, we should expect the Hessian to give a signature and we should trust the threshold step to remove insignificant structures. According to some, however, additional filtering steps should be taken. This will be studied further via the comparison of multiple threshold functions in NeoNEXUS.

An unfortunate factor in comparing NeoNEXUS with its predecessors is the limitations of the programming language. For example, NEXUS was written in C++ and NeoNEXUS in Julia, which means some high level factors may slow the code down. Numerically there should be no significant differences, but as far as computation time is concerned, Julia may be ever so slightly slower. This is also a tradeoff in abstraction - creating many functions and structures is very convenient for programmers reading the code, but in principle the most optimal possible code is written in assembly or even binary and every abstraction above it slows things down. This is why speed benchmarks with NEXUS for the time being is avoided.

6.3 Comparison with Sister Project

For the sake of transparency it should be noted that right now a parallel MMF/NEXUS project is in the works, spearheaded by Bram Alferink and supervised by Rien van de Weygaert. I am also involved in the project. That project is an implementation of MMF/NEXUS in Julia *directly*, that is to say, without the element of abstraction. That code was written precisely for cosmological use and is directed towards a userbase of cosmologists. The main goal is direct and convenient usage. My work on that project has been in suggesting improvements both for performance and readability as well as developing front end tools.

My primary accomplishments on that MMF/NEXUS project has been developing a GUI for simpler use by someone not familiar with Julia as well as working on a portable version of the program, precompiled and ready to use out of the box. The former is already done, although the library used for the GUI is being reconsidered due to causing issues with the latter. Portable versions of that project are underway for all platforms, hopefully meaning that users can begin using a version of MMF/NEXUS as soon as possible.

As noted above, this sister project is not a competitor to this one, since the target audience is vastly different and was made with a completely different design philosophy. That implementation lacks the NEXUS velocity family of programs and is essentially hard coded to function in a very specific way. We have reason to believe that way is close to optimal, but it is still preferable

that the standard NEXUS package is one with more options. That is why NeoNEXUS should, when finished, serve as a better replacement.

I should note, my work on that MMF/NEXUS implementation with Bram Alferink has been invaluable in my better understanding of the inner workings of the process, particularly during extended discussion and debate about implementation.

6.4 Further Planned Improvements

As previously stated, the NeoNEXUS project is still some way from being complete. The most glaring functional improvements to be made are on filters and thresholds. For one, additional filters and thresholds should be developed, at the very least to justify the abstraction already created. On a more user-based side, we also want there to be more than one example of how to create a new filter (or threshold) to make sure programmers extending the framework don't copy too much from a single instance.

As previously mentioned, an ideal version of NeoNEXUS would also have a portable version that can easily be run by any user, ideally with a GUI. Thankfully, the work and experience spent on the sister project to this one will be invaluable in creating NeoNEXUS portable versions and the GUI (at least for the base use case) is essentially directly reusable. This means even if not a part of NeoNEXUS yet, some of the steps towards the final application are already in motion.

7 Potential Use Cases

Previous sections have outlined the many improvements NeoNEXUS can bring to its original cosmological use case, namely a more modular and applicable environment for researchers in the field. For the remainder of this section, I will go over uses beyond cosmology, spanning into other fields as proposed potential projects. Keep in mind this is a very non-exhaustive list, showing the potential uses I have gleamed. The general STEM field is driven by the wisdom of the many and often unlikely tools are used by innovators in unexpected fields.

7.1 Astronomical Applications

The first and least generalized set use cases for NeoNEXUS is still within astronomy, but not precisely in cosmology. After all, the term "astrophysics" is more than a little broad. I will outline different fields in astronomy that will likely find use of NeoNEXUS. The use cases are ordered in descending order of physical scales, starting from just below cluster scale.

First, comes the field of galaxy formation and cloud collapse. The field large scale structure formation, as outlined in previous sections, focuses mostly on collapse along a single direction, because it is governed by the Zel'dovich approximation. However, going just below that, to the realm of baryonic matter collapsing into galaxies, we get much more complex interactions, particularly in the velocity field. While NEXUS already dealt with shears, things like vorticity are essentially excluded, due to the Zel'dovich approximation. However, on the scale of galaxy formation within clusters, vorticity become very relevant and NeoNEXUS allows for any user defined features, including ones based on vorticity.

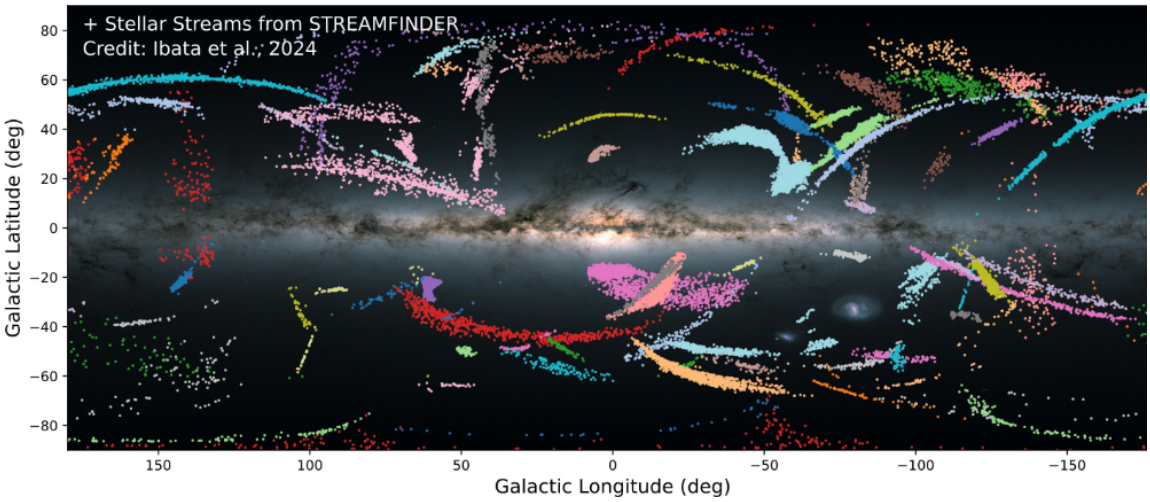

Figure 5: Stellar streams in the Milky Way shown using data from Ibata et al 2024[6]. The streams show merger events in the Milky way history. The individual stars are smeared across the entire night sky.

Going slightly lower, we have already formed galaxies. Given an arbitrary density field, which may have galaxies and galactic features present, if one is tasked with finding specific features (such as galactic spirals), this can prove cumbersome. NeoNEXUS can be used for this as well, although it may find itself outclassed by more specialized image processing tools that have been developed for the field precisely. Nevertheless, there is still a potential for use.

Moving to a lower scale we find ourselves in the field of Galactic Archaeology, or to be less facetious, detection of stellar streams. Researchers in the field of galactic archaeology try to identify objects which have merged into the Milky Way and are at this point unidentifiable, due to mixing with in situ stars. When plotted in the Milky Way, streams appear as long stretched lines along the galaxy. A good example is shown in Figure 5 using data from Ibata et al 2024[6]. Needless to say, identification of these streams using conventional methods is close to impossible. Different numerical methods have been developed, including integrals of motion grouping (more on this below), but among all of those, NeoNEXUS can prove to be one as well. Given information on individual stars and the knowledge that disrupted streams (roughly) exist on a part of a ring, a custom signature can be created and used.

Finally, we enter the realm of more esoteric spaces. While all examples so far have been dealing whith physical 3D space, nothing in NeoNEXUS enforces that. As long as a field is given and a signature defined, any dimensional field of arbitrary distances can be used. Therefore, here I will discuss a few (pseudo) phase spaces, which can be explored with some thought using NeoNEXUS.

The fundamental plane is a 2D slice in the parameter space of elliptical galaxies correlating effective radius, surface brightness, and velocity dispersion. In this space, galaxies occupy a relatively thin slab diagonally across the full space, making it a natural candidate for morphological analysis. Deviations from the fundamental plane can encode information about formation history, environment, or dynamical state. NeoNEXUS could be used in this parameter space to identify substructures, outliers, or coherent trends that may otherwise be ignored by traditional techniques.

An HR Diagram is a classic example of a two-dimensional pseudo–phase space, mapping stellar luminosity against effective temperature (or color). Stellar populations form highly structured sequences such as the main sequence, red giant branch, and white dwarf cooling tracks, all highly dependent on the metallicity of the population. By defining appropriate signatures, NeoNEXUS could be used to automatically identify evolutionary tracks, detect sub-populations, or quantify morphological differences between stellar samples.

Integrals of Motion space is a phase space representation constructed from conserved quantities in galactic merger events such as orbital energy and angular momentum. In this space, stars originating from the same disrupted progenitor tend to remain clustered long after they have become spatially mixed in real space (see Figure 4). This property has made integrals of motion space a powerful tool in galactic archaeology, for example by clustering algorithms. NeoNEXUS offers a novel approach via the definition of morphology-based signatures tailored to the expected geometry of merger debris in this space, potentially allowing for the systematic detection and classification of stellar streams and accretion remnants.

7.2 Engineering Applications

Finally leaving the realm of science, NeoNEXUS can find many use cases when applied to engineering, in particular material science and analysis. The approach to materials is split in two types. On one hand researchers working on new materials, especially on microscopic level can detect patterns and inhomogeneities along a large structure. For example a scan of a new alloy can be checked for specific underlying features. On the other hand, field engineers can use NeoNEXUS to test scans of materials to check for microscopic bends, cracks and fractures. This can easily be done by inverting the density file - looking for holes rather than overdensities.

7.3 Medical and Biomedical Applications

To conclude the proposed use cases, we would be remiss not to revisit the field MMF was originally inspired by - medical analysis. As already noted, medical imaging has been using scale space for morphological analysis since the 1990s[7]. The primary motivation in medicine is the sheer volume of data and the limited human resources available. These are competing forces - you want to screen as many people as possible for things like cancer, but you also can't spare the personnel to review every image. That is why medicine is also subject to Big Data and needs similar solutions. NeoNEXUS as a framework can already essentially be applied to medical scans to screen for node formations (cancer) and interconnected linear structures (cardio-vascular system). Furthermore, one can easily imagine a medical researcher defining other more complex features to find other important formations. Since NeoNEXUS permits the use of any auxiliary field (which in astronomical uses was velocity) one could use a second set of data (such as body temperature or pressure) to detect formations in alternative ways.

One important note for medicinal use is a huge difference in the purity versus completeness dilemma. This is a topic that arises particularly when dealing with AI - when probability is involved one has to decide if the detections should be more zealous. After stripping away all fancy metrics for false positives and true negatives, one reaches the final decision - would you want to know every detection is true at the cost of missing some or would you prefer to have false

detections. In the case of medicine thresholds need to be defined such that the set of detections is as complete as possible. At the end of all detections a human doctor will review the data and it is better for them to rule out a detection than for you to miss one. In this way, NeoNEXUS, along with other computational tools[12], will only act as a supliment to qualified personnel, never a replacement.

7.4 Future Extensions

As already noted at the beginning of this section, the aforementioned ideas I had where NeoNEXUS could be applied outside of cosmology. There are likely many other examples for use cases in various fields. At the end of the day, NeoNEXUS is a framework that is given to other experts to use as they find best. Future extensions can be made by developers in any field of science and beyond.

8 Conclusion and Recommendations

To conclude this report, I will give an overall summary of NeoNEXUS and its merits and finish off by giving my recommendations for future work with the project.

8.1 Summary of Contributions

NeoNEXUS is a framework that extends the uses of MMF/NEXUS into a much wider range of use cases. It utilizes the underlying robust analytical principles outlined in Section 3, while also improving on the flaws of the previous iterations listed in Section 4. It is implemented in a modular and generalized way in Julia as outlined in Section 5. It has a wide variety of use cases within astronomy as noted in Sections 6 and 7.1 as well as in fields beyond it, as outlined in the rest of Section 7. The framework can be expanded both for its core cosmological use, as well as for any particular field that may find use for it owing to the most notable merit of the framework - its generalized nature.

8.2 Recommendations for Future Work

Work on the NeoNEXUS framework has birthed the inspiration for a lot of future work. First, I will note that while building the structures in Julia, I noticed how naturally NeoNEXUS lends itself to Object Oriented Programming. Perhaps it is a bias on my end, since I have an extensive background in Java, but it does seem that the way Features work in the framework, an object with multiple methods and fields would be an excellent fit for the project. Unfortunately, Julia does not support OOP, likely due to performance issues associated with the model. I would certainly find it interesting to see the framework translated in OOP, preferably not in Java or Python, owing to the slow and heavy nature of both.

As for scientific work with the NeoNEXUS model, the opportunities are nearly endless. Currently some elements of MMF/NEXUS are present as they are due to precedent. A lot of future projects can focus on finding what the optimal filter type can be (Gaussian, sharp step function,

[12] To be explicit, this includes any AI tools.

etc) or what better thresholding methods could be used. NEXUS+ is an excellent update to NEXUS by simply taking the logarithm, so why not test other functions and transformations? Even the spacing of the scale space bins is there since MMF based on a motivation related to the human eye's discernment of scales. All of the aforementioned and more can be explored into larger works by simply replacing one component of NeoNEXUS and checking if the results are better or worse and how much. As far as future work goes, the sky is the limit.

Acknowledgments

I would like to thank my research supervisor, Rien van de Weygaert, for his guidance through this project as well as all other projects done in relation to this one. I want to thank my collaborator, Bram Alferink, for his help and consultation with matters of MMF/NEXUS and I am eagerly looking forward to our work on related projects. Finally, I would like to thank my father, Konstantin Spirov, for his consultation and aid both for this project and my overall software development.

References

[1] M. A. Aragón-Calvo, B. J. T. Jones, R. van de Weygaert, and J. M. van der Hulst. The multi-scale morphology filter: identifying and extracting spatial patterns in the galaxy distribution. , 474(1):315–338, October 2007.

[2] Marius Cautun, Rien van de Weygaert, and Bernard J. T. Jones. NEXUS: tracing the cosmic web connection. , 429(2):1286–1308, February 2013.

[3] Marius Cautun, Rien van de Weygaert, Bernard J. T. Jones, and Carlos S. Frenk. Evolution of the cosmic web. *Monthly Notices of the Royal Astronomical Society*, 441(4):2923–2973, Jul 2014.

[4] DESI Collaboration. The desi survey: Mapping tens of millions of galaxies across the cosmos. `https://www.desi.lbl.gov/the-desi-survey/`, 2025. Accessed 2025-01-04.

[5] European Space Agency (ESA). Gaia creates richest star map of our galaxy — nearly 1.7 billion stars. `https://www.esa.int/Science_Exploration/Space_Science/Gaia/Gaia_creates_richest_star_map_of_our_Galaxy_and_beyond`, 2025. Accessed 2025-01-04.

[6] Rodrigo Ibata, Khyati Malhan, Wassim Tenachi, Anke Ardern-Arentsen, Michele Bellazzini, Paolo Bianchini, Piercarlo Bonifacio, Elisabetta Caffau, Foivos Diakogiannis, Raphael Errani, Benoit Famaey, Salvatore Ferrone, Nicolas F. Martin, Paola di Matteo, Giacomo Monari, Florent Renaud, Else Starkenburg, Guillaume Thomas, Akshara Viswanathan, and Zhen Yuan. Charting the Galactic Acceleration Field. II. A Global Mass Model of the Milky Way from the STREAMFINDER Atlas of Stellar Streams Detected in Gaia DR3. , 967(2):89, June 2024.

[7] Yoshinobu Sato, Shin Nakajima, Nobuyuki Shiraga, Hideki Atsumi, Shigeyuki Yoshida, Thomas Koller, Guido Gerig, and Ron Kikinis. Three-dimensional multi-scale line filter for segmentation and visualization of curvilinear structures in medical images. *Medical Image Analysis*, 2(2):143–168, 1998.

[8] Ya. B. Zel'dovich. Gravitational instability: An approximate theory for large density perturbations. , 5:84–89, March 1970.